Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра программного обеспечения и технологий

Учебная дисциплина «Метрология, стандартизация и сертификация (в информационных технологиях)»

Отчет

по лабораторной работе №2
«Метрики Джилба»

Подготовил: Студенты гр. 410902
Кавцевич В. А.
Габрукович Р. С.

Проверил: Болтак С. В.

Минск 2026

Цель: Изучить метрики сложности потока управления.

Задание: Подготовить код анализируемой программы для расчета метрики Джилба. В коде для расчета метрики Джилба должны присутствовать все операторы цикла заданного языка, а также операторы ветвления, включая оператор множественного выбора. Написать программу-парсер. Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять подготовленную программу на заданном языке. Результат работы парсера: вывод на экран абсолютной сложности, относительной сложности и максимального уровня вложенности.

Код анализируемой программы на JavaScript:

```
const students = [
    { name: "Иванов", marks: { math: 9, physics: 7, history: 4 } },
    { name: "Петров", marks: { math: 3, physics: 2, history: 4 } },
    { name: "Сидорова", marks: { math: 10, physics: 9, history: 8 } },
    { name: "Кузнецов", marks: { math: 5, physics: 6, history: 5 } },
];
function average(marks) {
    let sum = 0;
    let count = 0;
    for (const subject in marks) {
        sum += marks[subject];
        count++;
    }
    return count > 0 ? sum / count : 0;
}
function grade(avg) {
    let level = Math.floor(avg);
    let result;
    switch (level) {
        case 10:
        case 9:
            result = "отлично";
            break;
        case 8:
        case 7:
            result = "хорошо";
            break;
        case 6:
        case 5:
        case 4:
            result = "удовлетворительно";
            break;
        default:
```

```
            result = "неудовлетворительно";
    }
    return result;
}
function countDebtors(list) {
    let debtors = 0;
    for (const student of list) {
        const avg = average(student.marks);
        if (avg < 4) {
            debtors++;
        }
    }
    return debtors;
}
function maxMark(list, subject) {
    let best = 0;
    for (const student of list) {
        if (student.marks[subject] > best) {
            best = student.marks[subject];
        }
    }
    return best;
}
let debtors = 0;
for (const student of students) {
    const avg = average(student.marks);
    if (avg >= 4) {
        console.log(student.name + ": " + grade(avg));
    } else {
        debtors++;
        for (let attempt = 1; attempt <= 3; attempt++) {
            if (attempt === 3) {
                console.log(student.name + ": комиссия");
            } else if (attempt === 1) {
                console.log(student.name + ": пересдача");
            }
        }
    }
}
console.log("Всего должников: " + debtors);
let i = 0;
while (i < students.length && students[i].marks.math < 10) {
    i++;
}
if (i < students.length) {
```

```
    console.log("Отличник по математике: " + students[i].name);
} else {
    console.log("Отличников по математике нет");
}
let attempts = 0;
do {
    attempts++;
} while (attempts < debtors);
console.log("Попыток сдачи: " + attempts);
console.log("Лучший балл по физике: " + maxMark(students, "physics"));
async function printNames() {
    for await (const s of students) {
        console.log(s.name);
    }
}
printNames();
let check = countDebtors(students);
if (check === debtors) {
    console.log("Проверка пройдена");
} else {
    console.log("Ошибка: " + check + " != " + debtors);
}
```

Результаты расчета метрик Джилба представлены в таблице 1.

Таблица 1 – Результаты расчета метрик Джилба.

№	Условный оператор	№	Оператор программы
1	for (const subject in marks)	1	const students = […]
2	count > 0 ? sum / count : 0	2	let sum = 0
3	case 10: case 9:	3	let count = 0
4	case 8: case 7:	4	for (const subject in marks)
5	case 6: case 5: case 4:	5	sum += marks[subject]
6	for (const student of list)	6	count++
7	if (avg < 4)	7	return count > 0 ? sum / count : 0
8	for (const student of list)	8	let level = Math.floor(avg)
9	if (student.marks[subject] > best)	9	let result
10	for (const student of students)	10	case 10: case 9:
11	if (avg >= 4)	11	result = "отлично"
12	for (let attempt = 1; attempt <= 3; attempt++)	12	case 8: case 7:
13	if (attempt === 3)	13	result = "хорошо"

№	Условный оператор	№	Оператор программы
14	else if (attempt === 1)	14	case 6: case 5: case 4:
15	while (i < students.length && students[i].marks.math < 10)	15	result = "удовлетворительно"
16	if (i < students.length)	16	result = "неудовлетворительно"
17	do … while (attempts < debtors)	17	return result
18	for await (const s of students)	18	let debtors = 0
19	if (check === debtors)	19	for (const student of list)
		20	const avg = average(student.marks)
		21	if (avg < 4)
		22	debtors++
		23	return debtors
		24	let best = 0
		25	for (const student of list)
		26	if (student.marks[subject] > best)
		27	best = student.marks[subject]
		28	return best
		29	let debtors = 0
		30	for (const student of students)
		31	const avg = average(student.marks)
		32	if (avg >= 4)
		33	console.log(student.name + ": " + grade(avg))
		34	debtors++
		35	for (let attempt = 1; attempt <= 3; attempt++)
		36	if (attempt === 3)
		37	console.log(student.name + ": комиссия")
		38	else if (attempt === 1)
		39	console.log(student.name + ": пересдача")
		40	console.log("Всего должников: " + debtors)
		41	let i = 0
		42	while (i < students.length && students[i].marks.math < 10)

№	Условный оператор	№	Оператор программы
		43	i++
		44	if (i < students.length)
		45	console.log("Отличник по математике: " + students[i].name)
		46	console.log("Отличников по математике нет")
		47	let attempts = 0
		48	do … while (attempts < debtors)
		49	attempts++
		50	console.log("Попыток сдачи: " + attempts)
		51	console.log("Лучший балл по физике: " + maxMark(students, "physics"))
		52	for await (const s of students)
		53	console.log(s.name)
		54	printNames()
		55	let check = countDebtors(students)
		56	if (check === debtors)
		57	console.log("Проверка пройдена")
		58	console.log("Ошибка: " + check + " != " + debtors)
CL=19		N=58	

Базовые метрики:
Абсолютная сложность программы CL = 19.
Относительная сложность программы cl = CL/N = 19/58 = 0,328.
Расширенная метрика:
Максимальный уровень вложенности CLI = 4.

Программа-парсер написана на языке Python с использованием библиотеки Tkinter для графического интерфейса. Результатом работы программы является вывод таблицы ветвлений, учтённых в CL, таблицы операторов программы, а также результата расчёта метрик Джилба: абсолютной сложности CL, относительной сложности cl и максимального уровня вложенности CLI. Результат работы программы представлен на рисунках 1–4.

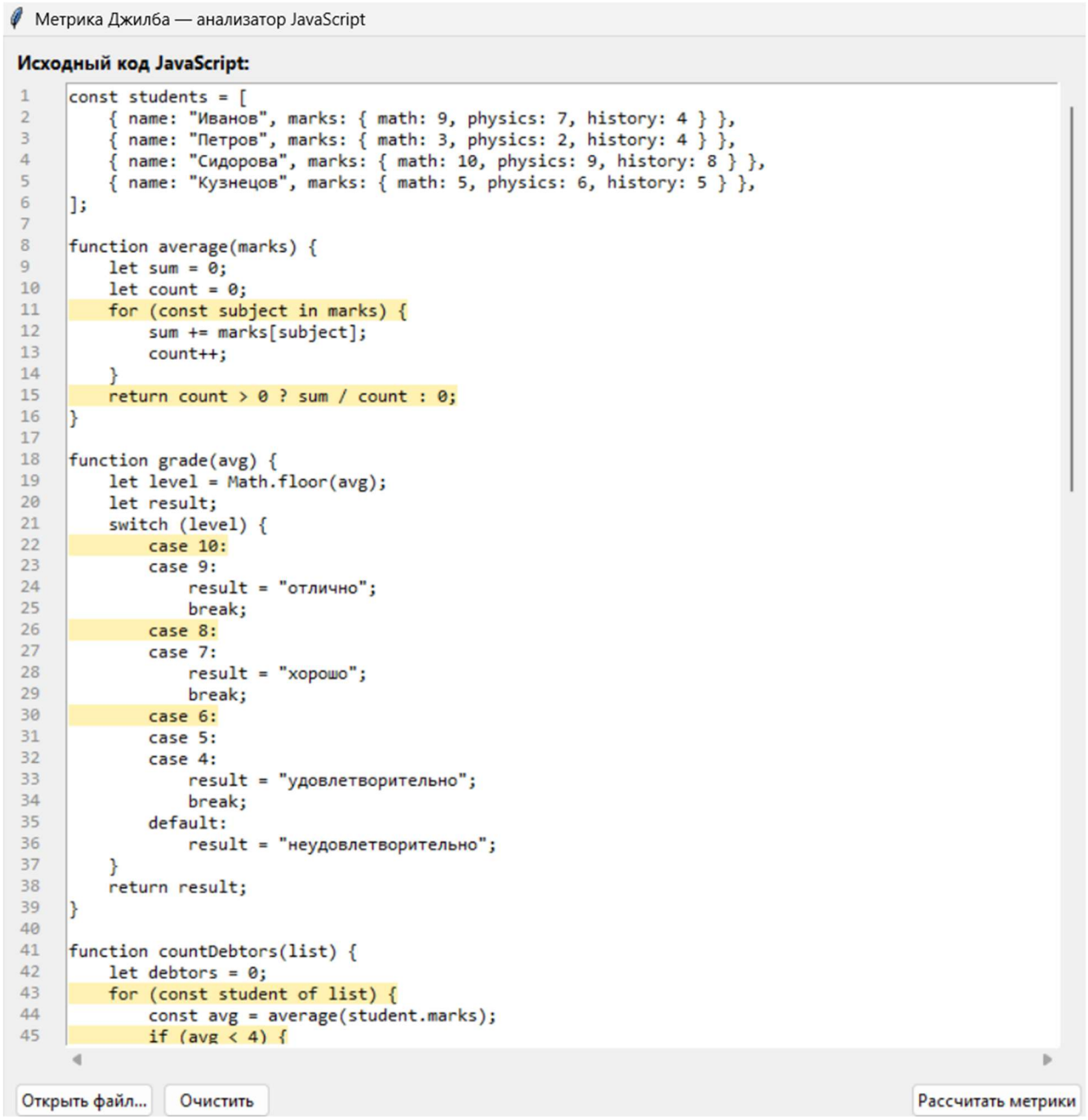


Рисунок 1 – Главное окно с загруженным test.js и подсвеченными ветвлениями

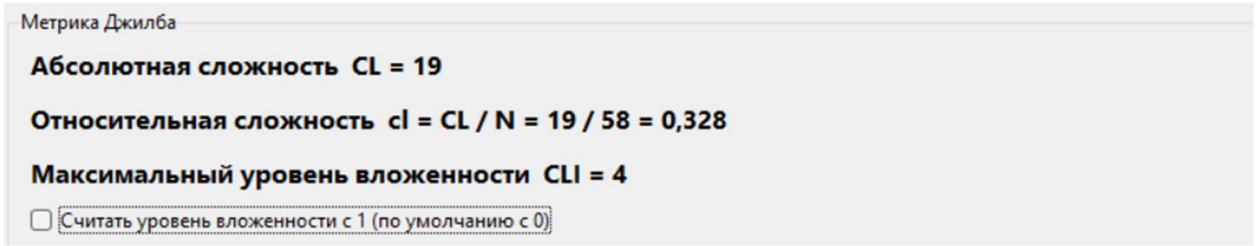


Рисунок 2 – Панель «Метрика Джилба» со значениями CL, cl, CLI

Ветвления, учтённые в CL

№	Строка	Вид	Уровень	Фрагмент кода
1	11	for..in	0	for (const subject in marks) {
2	15	?: (тернарный)	0	return count > 0 ? sum / count : 0;
3	22	switch-case	0	case 10:
4	26	switch-case	1	case 8:
5	30	switch-case	2	case 6:
6	43	for..of	0	for (const student of list) {
7	45	if	1	if (avg < 4) {
8	54	for..of	0	for (const student of list) {
9	55	if	1	if (student.marks[subject] > best) {
10	63	for..of	0	for (const student of students) {
11	65	if	1	if (avg >= 4) {
12	69	for	2	for (let attempt = 1; attempt <= 3; attempt++) {
13	70	if	3	if (attempt === 3) {
14	72	else if	4	} else if (attempt === 1) {
15	82	while	0	while (i < students.length && students[i].marks.math < 10) {
16	85	if	0	if (i < students.length) {
17	92	do..while	0	do {
18	100	for await..of	0	for await (const s of students) {
19	107	if	0	if (check === debtors) {

Рисунок 3 – Таблица «Ветвления, учтённые в CL»

Операторы программы (знаменатель cl)

№	Строка	Вид оператора
1	1	const
2	9	let
3	10	let
4	11	for..in
5	12	выражение
6	13	выражение
7	15	return
8	19	let
9	20	let
10	22	case → if
11	24	выражение
12	26	case → if
13	28	выражение

14	30	case → if
15	33	выражение
16	36	выражение
17	38	return
18	42	let
19	43	for..of
20	44	const
21	45	if
22	46	выражение
23	49	return
24	53	let
25	54	for..of
26	55	if
27	56	выражение
28	59	return
29	62	let
30	63	for..of
31	64	const
32	65	if
33	66	выражение
34	68	выражение
35	69	for
36	70	if
37	71	выражение
38	72	else if
39	73	выражение
40	79	выражение
41	81	let
42	82	while
43	83	выражение
44	85	if
45	86	выражение
46	88	выражение
47	91	let
48	92	do..while
49	93	выражение
50	95	выражение
51	97	выражение
52	100	for await..of
53	101	выражение
54	104	выражение
55	106	let
56	107	if
57	108	выражение
58	110	выражение

Рисунок 4 – Таблица «Операторы программы (знаменатель cl)»

Схема алгоритма работы программы изображена на рисунке 5.

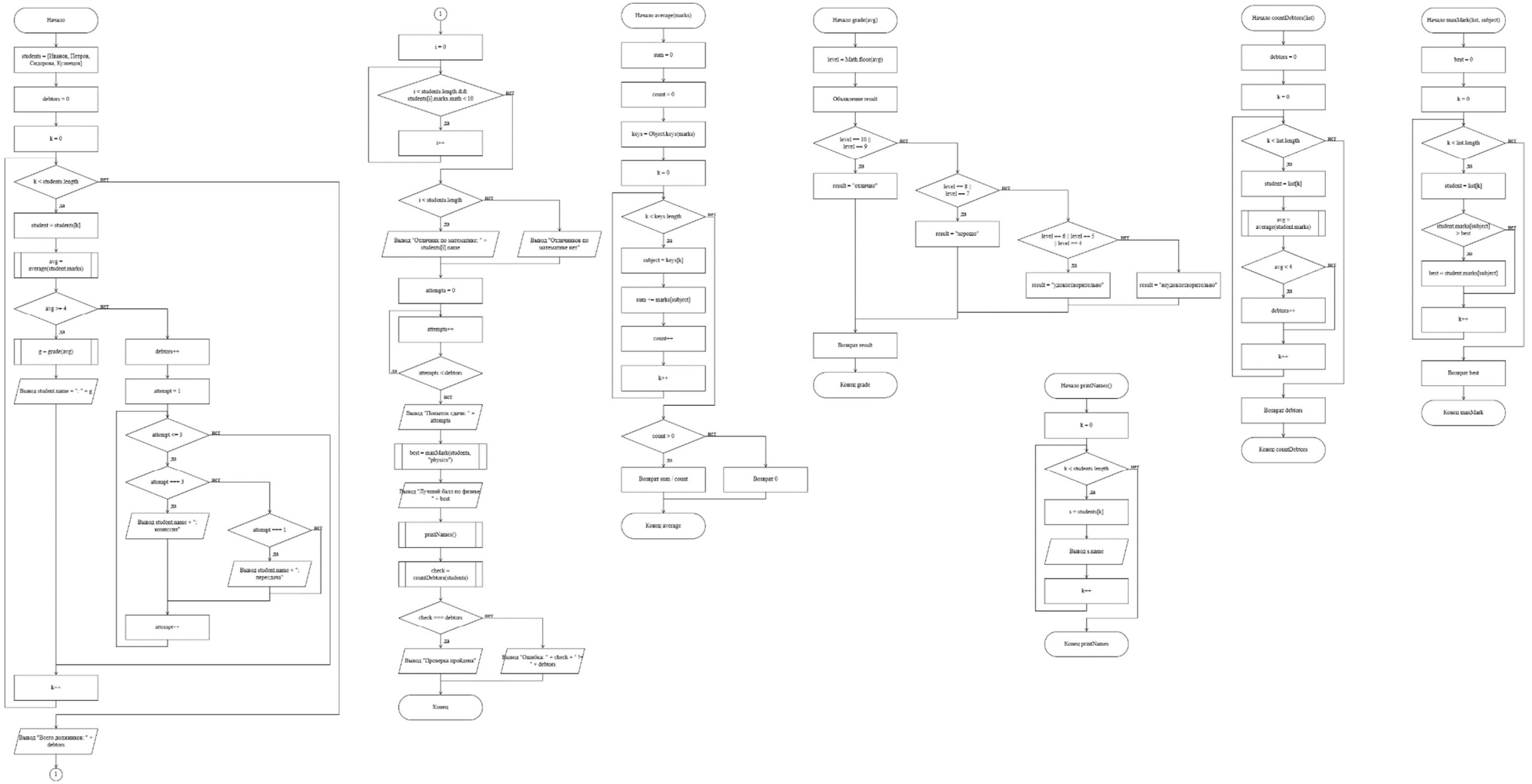


Рисунок 5 – Схема алгоритма работы программы